

Accelerating GRPO Post-Training with Completion Pruning

An implementation and analysis of CPPO for Qwen3-0.6B on DAPO-Math-17k

Yusuf Saifutdinov

September 27, 2026

Abstract

Group Relative Policy Optimization (GRPO) has become the workhorse of reasoning-oriented post-training, but it is expensive: every question is answered G times, and every one of those G completions is pushed through a forward and a backward pass. Completion Pruning Policy Optimization (CPPO) observes that a completion’s contribution to the policy gradient is proportional to the magnitude of its group-relative advantage, and therefore back-propagates through only the $k = \lfloor G(1 - P) \rfloor$ completions with the largest $|A_i|$. This report presents a from-scratch implementation of GRPO and CPPO on top of TRL for Qwen3-0.6B trained on DAPO-Math-17k, evaluated with `lm-evaluation-harness` on GSM8K, Minerva MATH and AIME 2024. We describe both algorithms, show how CPPO’s *dynamic completion allocation* maps exactly onto TRL’s batching parameters without modifying TRL, and give a cost model that predicts CPPO’s end-to-end speedup from a single measured quantity: the share of training time spent in the update stage. The central finding is that CPPO’s accuracy is essentially free — pruning discards completions that carry almost none of the batch’s learning signal — but its *speedup is bounded by Amdahl’s law* on the rollout stage, so the published $8.32\times$ figures are reachable only in configurations where the update stage dominates.

Contents

1	Introduction	2
2	Background: GRPO	2
2.1	Objective	2
2.2	Rule-based reward	3
2.3	Where the time goes	3
3	CPPO: Completion Pruning Policy Optimization	3
3.1	Which completions actually matter	3
3.2	The pruned objective	4
3.3	Dynamic completion allocation	4
3.4	Algorithm	4
4	Implementation	4
4.1	Selection	5
4.2	Mapping dynamic allocation onto TRL	5
4.3	Where pruning is applied, and why	6
4.4	Correctness details that are easy to get wrong	6
4.5	Accelerating the rollout stage	7
5	Experimental setup	7

6	Results	8
6.1	Update-stage scaling, measured on Qwen3-0.6B	8
6.2	End-to-end pipeline validation	9
6.3	What is not measured here	9
6.4	Tables	10
7	Analysis	10
7.1	A cost model for CPPO’s speedup	10
7.2	Why accuracy survives	11
7.3	Memory	12
8	Limitations and reproducibility	12
9	Conclusion	13

1 Introduction

Reinforcement learning with verifiable rewards has driven most of the recent progress in mathematical reasoning, code generation and long-horizon instruction following [1, 11]. The dominant algorithm family is PPO [8] and its critic-free descendant GRPO [9], which replaces the learned value function with a baseline estimated from a *group* of completions sampled for the same question.

Removing the critic removes one model from GPU memory, but GRPO remains costly for three reasons:

1. **Multiple models resident at once.** The policy, and — when a KL penalty is used — a frozen reference model, plus optimiser state for the policy.
2. **A slow autoregressive rollout.** Each optimiser step needs G sampled completions *per question*, and sampling is memory-bandwidth bound rather than compute bound.
3. **A wide update stage.** Every one of those G completions is pushed through a forward and a backward pass of the policy.

CPPO [5] attacks the third cost. This report answers the question posed by the task directly: *how accurate is CPPO, and how does it influence RL training acceleration?* Section 2 and Section 3 describe the two algorithms, Section 4 documents the implementation and the places where a faithful mapping onto TRL required judgement, Section 5–6 give the experimental protocol and results, and Section 7 develops the cost model that explains what the numbers mean.

2 Background: GRPO

2.1 Objective

For a question q drawn from the data distribution $P(Q)$, the old policy $\pi_{\theta_{\text{old}}}$ samples a group of G completions $\{o_1, \dots, o_G\}$. GRPO maximises

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(o|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left(\rho_{i,t}(\theta) A_i, \right. \right. \right. \\ \left. \left. \left. \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_i \right) - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right\} \right], \quad (1)$$

where $\rho_{i,t}(\theta) = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ is the token-level importance ratio and the advantage is the group-standardised reward

$$A_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}. \quad (2)$$

Crucially, A_i is constant across the tokens of completion i : the whole completion is weighted by one scalar.

2.2 Rule-based reward

Following the CPPO paper and common practice for verifiable domains, the reward is the sum of a format term and an accuracy term, $r_i = R_{\text{format}}(o_i) + R_{\text{accuracy}}(o_i)$, with $R_{\text{format}} \in \{0, 1\}$ and $R_{\text{accuracy}} \in \{0, 2\}$. No reward model is trained. Our implementation checks accuracy with symbolic equivalence (`math_verify`) rather than string matching, so $\frac{1}{2}$, 0.5 and `\frac{1}{2}` all score alike.

2.3 Where the time goes

Let B be the number of questions per optimiser step. A step costs

$$T_{\text{step}} = \underbrace{B \cdot G \cdot c_{\text{gen}}}_{\text{rollout}} + \underbrace{B \cdot G \cdot (n_{\text{fwd}} c_{\text{fwd}} + c_{\text{bwd}})}_{\text{update}}, \quad (3)$$

where n_{fwd} counts the forward passes over completions: one for the policy, plus one for the reference model when $\beta \neq 0$, plus one for the old policy when the samples are off-policy. In DeepSeekMath’s configuration with $G = 64$ that is 192 forward passes per question [5]. Both terms scale linearly in G , which is what makes larger groups — and hence better baselines — so expensive.

3 CPPO: Completion Pruning Policy Optimization

3.1 Which completions actually matter

Differentiating Eq. (1) and dropping the KL term, which Hu et al. [3] show is not what shapes reasoning behaviour, gives

$$\nabla_\theta \mathcal{J}(\theta) \approx \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \underbrace{\rho_{i,t}(\theta)}_{\text{post-forward}} \cdot \underbrace{A_i}_{\text{pre-forward}} \cdot \underbrace{\nabla_\theta \log \pi_\theta(o_{i,t} \mid q, o_{i,<t})}_{\text{post-forward}} \right]. \quad (4)$$

Three factors must all be non-negligible for a completion to move the policy. Two of them are only knowable *after* the policy forward pass — but A_i is known *before* it, as soon as the group’s rewards have been scored. That asymmetry is the entire idea: a completion whose $|A_i|$ is near zero can be identified and discarded before any expensive computation is spent on it.

The paper also gives an interpretation of *which* completions get discarded. With a two-part reward, the four outcome types are:

Format	Answer	Typical $ A $	Training signal
correct	correct	large	“do this” — clean positive signal
incorrect	incorrect	large	“avoid this” — clean negative signal
correct	incorrect	small	ambiguous; rewards form over substance
incorrect	correct	small	ambiguous; rewards luck over method

Pruning by $|A_i|$ therefore removes precisely the partially-correct completions whose gradient is both weak and semantically muddled, which is why CPPO can *improve* accuracy rather than merely preserve it.

3.2 The pruned objective

CPPO adds a selection constraint to Eq. (1). To keep every data-parallel device processing an identical number of completions — otherwise the slowest device sets the pace, the “bucket effect” — the constraint is expressed as a per-group top- k rather than a global threshold:

$$k = \lfloor G \cdot (1 - P) \rfloor, \quad P \in [0, 1), \quad (5)$$

$$\mathcal{I} = \{ i \in \{1, \dots, G\} \mid |A_i| \text{ is among the top } k \text{ values} \}, \quad (6)$$

$$\mathcal{J}_{\text{CPPO}}(\theta) = \mathbb{E}_{q, \{o_i\}} \left[\frac{1}{k} \sum_{i \in \mathcal{I}} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left(\rho_{i,t}(\theta) A_i, \right. \right. \right. \quad (7)$$

$$\left. \left. \left. \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_i \right) - \beta \mathbb{D}_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}] \right\} \right].$$

Note the normaliser: $1/k$, not $1/G$. Pruning must not shrink the gradient magnitude, or it would silently act as a learning-rate decay.

3.3 Dynamic completion allocation

Pruning leaves $G - k$ completion slots empty in the update stage, so the accelerator runs at a fraction of its capacity. CPPO refills those slots with completions sampled for *additional* questions. Per generation round the device then handles

$$m = \lfloor G/k \rfloor \quad (8)$$

times as many questions, each still contributing k retained completions, so the post-pruning batch is again the full width the hardware can hold. Total rollout work over an epoch is unchanged — the same questions are still answered G times — but the epoch now needs m times fewer optimiser steps, and each of those steps is as wide as the baseline’s.

3.4 Algorithm

Steps 1–3 of the loop are identical to GRPO. The only changes are the top- k selection and the m -fold widening of the question batch.

4 Implementation

The implementation is a small package layered on TRL [10] rather than a fork of it, so the GRPO baseline is provably stock TRL:

Algorithm 1: CPPO training step (with dynamic completion allocation)

Input: policy π_θ , group size G , pruning rate P , questions per round B , reward function $r(\cdot)$

```
1  $k \leftarrow \lfloor G(1 - P) \rfloor$ ;  $m \leftarrow \lfloor G/k \rfloor$ ;
2 for each generation round do
3   Sample  $m \cdot B$  questions  $\{q_j\}$ ; // allocation:  $m \times$  more questions
4   foreach question  $q_j$  do
5     Sample  $G$  completions  $\{o_{j,1}, \dots, o_{j,G}\} \sim \pi_{\theta_{\text{old}}}(\cdot | q_j)$ ;
6     Score rewards  $r_{j,i} \leftarrow R_{\text{format}}(o_{j,i}) + R_{\text{accuracy}}(o_{j,i})$ ;
7      $A_{j,i} \leftarrow (r_{j,i} - \text{mean}_i r_{j,\cdot}) / \text{std}_i r_{j,\cdot}$ ; // pre-forward information
8      $\mathcal{I}_j \leftarrow \text{top-}k(\{A_{j,i}\}_{i=1}^G)$ ; // pruning
9   end
10   $\mathcal{B} \leftarrow \bigcup_j \{o_{j,i} : i \in \mathcal{I}_j\}$ ; //  $m \cdot B \cdot k \approx B \cdot G$  completions
11  foreach micro-batch of  $\mathcal{B}$  do
12    Forward and backward  $\pi_\theta$  on the micro-batch only; accumulate gradients;
13  end
14  Optimiser step;
15 end
```

Module	Responsibility
cppo/pruning.py	Top- k selection by $ A $; pruning diagnostics
cppo/geometry.py	Pruning rate $\rightarrow$ TRL batch parameters
cppo/trainer.py	ProfiledGRPOTrainer, CPPOTrainer
cppo/profiling.py	Stage timing and peak-memory tracking (shared)
cppo/rewards.py	Format and accuracy rewards
cppo/data.py	DAPO-Math-17k loading and prompt templating
cppo/train.py	Training entry point
cppo/evaluate.py	lm-evaluation-harness wrapper
cppo/report.py	Renders result tables for this document

4.1 Selection

The retention mask is built per group with a *stable* descending sort of $|A|$, so ties resolve to the lowest index and every data-parallel rank selects the same completions from the same advantages without communicating. Keeping exactly k per group — rather than thresholding $|A_i| \geq \gamma$ — is what makes the retained count identical on every device, which is the requirement Eq. (5) exists to satisfy.

4.2 Mapping dynamic allocation onto TRL

TRL derives its rollout batch from three knobs:

$$\begin{aligned} \text{generation_batch_size} &= \text{per_device_train_batch_size} \\ &\quad \times \text{num_processes} \times \text{steps_per_generation}, \end{aligned}$$

samples $\text{generation_batch_size}/G$ distinct questions per round, and splits the result into $\text{steps_per_generation}$ optimiser micro-batches.

The useful observation is that dynamic completion allocation requires *no changes to TRL’s batching code at all*. Multiplying $\text{per_device_train_batch_size}$ by m makes TRL sample m times more questions per round; pruning then shrinks each micro-batch back to exactly the

baseline width. Concretely, for the configuration used here ($G = 8$, baseline micro-batch 16, `steps_per_generation = 8`):

Run	P	k	m	Questions/round	Update micro-batch
GRPO baseline	0.000	8	1	16	16
CPPO $P = 0.50$	0.500	4	2	32	16
CPPO $P = 0.75$	0.750	2	4	64	16
CPPO $P = 0.875$	0.875	1	8	128	16
CPPO $P = 0.75$, no alloc.	0.750	2	1	16	4

The update stage therefore sees identical tensor shapes in every allocated configuration — identical memory, identical kernel occupancy — while covering m times more questions per optimiser step. The last row is the paper’s “+ Completion Pruning” ablation: pruning without refilling, which leaves three quarters of the batch empty.

4.3 Where pruning is applied, and why

The CPPO paper prunes before the policy, reference *and* old-policy forward passes. TRL computes rewards — and hence advantages — *after* the reference and old-policy log-probabilities, so a literal “prune first” ordering would require reimplementing a 600-line private method against internal TRL state, which is both fragile across versions and hard to audit.

Instead this implementation prunes at the end of the rollout, immediately before the policy forward/backward pass, and adopts a configuration in which the auxiliary passes *do not exist*:

- $\beta = 0$: no reference model, so no reference forward pass. This is TRL’s own default, matches Dr. GRPO’s recommendation [6], and is exactly the KL-free approximation from which CPPO derives its pruning criterion in Eq. (4). It also removes a whole model from GPU memory.
- $\mu = 1$ iteration with generation aligned to the optimiser step: the samples are on-policy, so TRL skips the old-policy forward pass.
- vLLM importance-sampling correction disabled: otherwise TRL recomputes old-policy log-probabilities to correct the sampler/trainer mismatch.

Under these settings the only pass over completions is the policy’s, and pruning before it reproduces Eq. (7) exactly. When a user overrides them, `CPPOTrainer` emits a warning naming each unpruned forward pass: training remains correct, but the measured speedup becomes a lower bound. Making that trade-off explicit — rather than silently reporting a smaller number — is the point.

4.4 Correctness details that are easy to get wrong

Loss normalisation. TRL’s default `dapo` loss divides by `num_items_in_batch`, the count of loss-carrying tokens in the whole generation batch. Pruning removes tokens; leaving the pre-pruning count in place would scale the loss — and therefore the effective learning rate — by k/G . The trainer recomputes the normaliser over the retained completions. This is the implementation-level counterpart of the $1/k$ in Eq. (7).

Group locality. Pruning is per question, so a group must live entirely on one device. The trainer validates that the local generation batch is a whole multiple of G and fails loudly otherwise, rather than silently pruning across a group boundary.

Degenerate groups. When all G completions of a question receive the same reward, the standard deviation is zero and every advantage is zero: the group contributes no gradient whatsoever, yet still costs a full forward and backward pass. The trainer logs the fraction of such groups, and offers an optional `drop_zero_advantage` mode that removes them outright. That mode is exact only when $\beta = 0$ and, because it makes the retained count data-dependent, is restricted to single-process runs — unequal batch sizes across ranks would deadlock a collective operation.

4.5 Accelerating the rollout stage

The task asks for acceleration of *both* stages. CPPO addresses the update stage only: advantages do not exist until sampling has finished, so no completion-pruning method can shrink the rollout. The three costs named in the problem statement map onto this pipeline as follows.

Simultaneous storage of multiple LLMs. Setting $\beta = 0$ removes the reference model entirely — no second set of weights and no reference forward pass. vLLM *sleep mode* releases the sampler’s KV cache and weights between rollouts, so the sampler and the optimiser never hold GPU memory at the same time. Optional LoRA (`use_peft`) reduces optimiser state from all 0.6B parameters to the adapter’s. It is worth stating plainly that $\beta = 0$ is a memory optimisation and not merely a modelling choice: it is the single largest memory saving in the pipeline, and it is also what makes pruning-before-every-forward-pass exact (Section 4.3).

Slow autoregressive rollout. Generation uses vLLM [4] with PagedAttention and continuous batching, in *colocate* mode so the sampler shares the training process rather than requiring a second device. CPPO’s dynamic completion allocation helps here too, which is easy to miss: enlarging the generation batch by $m = \lfloor G/k \rfloor$ gives vLLM more sequences to batch per decoding step, which is precisely what continuous batching needs to reach high throughput. Allocation is usually described as an update-stage trick; it is also a rollout-stage one.

Backpropagation updates. This is CPPO itself — back-propagating through k of G completions selected by $|A_i|$ — together with gradient checkpointing and bfloat16.

What the pipeline does *not* do is reduce the number of tokens generated. That is the remaining lever, and Section 7 explains why it is now the one that matters most.

5 Experimental setup

Policy model	Qwen3-0.6B [7]
Training data	<code>open-r1/DAP0-Math-17k-Processed</code> (config <code>en</code>)
Training subset	8,192 problems, one epoch
Reward	<code>format ({0,1}) + accuracy ({0,2}, <code>math_verify</code>)</code>
Prompt	chat template, “reason step by step, answer in <code>\boxed{}</code> ”
Group size G	8
Max completion len.	1024 tokens
Sampling	temperature 1.0, top- p 1.0
Objective	<code>dapo</code> loss, $\epsilon = 0.2$, $\epsilon_{\text{high}} = 0.28$, $\beta = 0$
Optimiser	AdamW, lr 10^{-6} , constant with 10 warmup steps, grad-clip 0.2
Rollout backend	vLLM [4], <i>colocate</i> mode
Evaluation	<code>lm-evaluation-harness</code> [2]: <code>gsm8k</code> (5-shot), <code>minerva_math</code> (4-shot), <code>aime24</code> (0-shot)

Protocol. Every run consumes the *same 8,192 problems in the same order*. Because dynamic allocation covers m times more questions per optimiser step, CPPO completes that epoch in m times fewer steps. Fixing the data rather than the step count is what makes a wall-clock comparison meaningful; fixing the step count instead would let CPPO see m times more data and confound speed with sample efficiency.

The results tables therefore report *questions per second* as the headline metric rather than raw wall clock. Throughput is comparable under either protocol: it credits an algorithm for covering the same training data in less time, and never for covering more data in the same time.

Prompt/evaluation alignment. The policy is trained with a chat template and a `\boxed{}` answer convention. Evaluation passes `--apply_chat_template` to `lm_eval` along with the training system prompt, so the two prompt formats agree. Both `minerva_math` and `aime24` extract the answer from `\boxed{}`, while `gsm8k`’s `flexible-extract` filter takes the last number, so the training format transfers to all three benchmarks without a task-specific adapter.

6 Results

6.1 Update-stage scaling, measured on Qwen3-0.6B

Apple M1 Pro (14-core GPU, 16 GB unified memory), PyTorch MPS backend, bfloat16, gradient checkpointing enabled, completion length 128 tokens, three timed steps after one warm-up. One group of $G = 8$ completions is pruned to k and the resulting forward, backward and optimiser step is timed; no generation and no reward model are involved, so the measurement isolates exactly what CPPO changes.

A least-squares fit over the eight points (Figure 1) gives

$$T(k) = 0.3360 \text{ s} + 0.5163 \text{ s} \cdot k, \quad R^2 = 0.99982,$$

so the update stage is linear in the number of retained completions to within measurement noise. The intercept is the batch-independent cost — the AdamW update over all 0.6B parameters, kernel launches, synchronisation — and amounts to 7.5% of the baseline step. Two consequences follow.

Pruning alone cannot reach G/k . At $P = 0.75$ the measured speedup is $3.23\times$ rather than $4\times$, because the fixed cost is now amortised over a quarter of the work. The update-stage ceiling as $k \rightarrow 0$ is $T(8)/0.3360 = 13.3\times$.

Dynamic allocation recovers that loss. Refilling the batch with completions from $m = \lfloor G/k \rfloor$ questions restores full batch width, so the fixed cost is amortised as well as the baseline’s while m times more questions are covered: $4.01\times$ at $P = 0.75$ and $8.02\times$ at $P = 0.875$, against $3.23\times$ and $5.39\times$ for pruning alone (Figure 2). This reproduces, on different hardware and a different model, the mechanism behind the paper’s component ablation ($1.23\times$ for pruning alone, $1.65\times$ once allocation is added), and it is why Section 4.2 treats allocation as central rather than as a detail.

The two curves cross at $k = 3$, where $k \nmid G$ leaves the refilled batch holding six of eight slots. Pruning rates for which k divides G — $P \in \{0.5, 0.75, 0.875\}$ at $G = 8$ — are the ones worth configuring.

Memory is deliberately omitted from the benchmark table: the MPS backend exposes only an instantaneous allocation figure rather than a high-water mark, so it cannot report a trustworthy activation peak. Peak-memory measurement requires the CUDA path, which `cppo/profiling.py` uses when a CUDA device is present.

6.2 End-to-end pipeline validation

This is a plumbing check, not a performance claim. It exercises the whole pipeline — dataset, chat template, rollout, `math_verify` rewards, group-relative advantages, pruning, loss, optimiser step, profiling — on the actual model and the actual training corpus, and confirms that CPPO changes what it is meant to change and nothing else. Apple M1 Pro, CPU, float32, `configs/smoke_cpu.yaml`: $G = 4$, 32-token completions, two optimiser steps.

The resulting run comparison is the training-performance table in Section 6.4. Three things are confirmed and one is not.

Dynamic allocation works as designed. CPPO consumed four questions to GRPO’s two in the same wall clock: a $2.01\times$ throughput gain against a predicted $m = \lfloor G/k \rfloor = 2$. Both runs took the same number of optimiser steps on the same update-stage micro-batch width; CPPO simply covered twice the data per step. Update time was essentially unchanged (516s against 526s) while rollout grew (28.7s against 21.1s) — exactly the trade the method makes.

Pruning retains what it claims. The CPPO run logged `cpo/retention` = 0.5 throughout: $k = 2$ of $G = 4$ completions reached the policy backward pass.

The stage split is measured, not assumed. Rollout and update times partition the training step additively, produced by the same code in both runs.

Nothing about learning is confirmed. With 32-token completions Qwen3-0.6B emits no `\boxed{}` answer, so every reward was zero, every group had zero reward variance, and every advantage was therefore zero — the run logs this as `cpo/frac_degenerate_groups` = 1. The gradient is zero and the model does not move. That is a property of the deliberately tiny configuration, not of the pipeline: the unit tests in `tests/test_trainer_smoke.py` use a reward that varies within a group and assert that CPPO back-propagates through exactly the retained subset.

A caution about f for this run. The update stage accounts for 96% of the measured step time here, which would imply a very high Amdahl ceiling. That figure does not generalise: on CPU a 32-token rollout is cheap while a float32 backward pass is extravagant. On the reference setup — A100, bfloat16, vLLM rollout, 1024-token completions — the split is far less update-heavy, which is exactly why Section 7 insists that f be measured per configuration rather than assumed.

The evaluation pipeline was exercised too. Running `cpo.evaluate` against the untrained Qwen3-0.6B on five GSM8K problems (CPU, float32, 256 generated tokens) returns `exact_match,flexible-extract` = 0.20 and `exact_match,strict-match` = 0.00. Five problems say nothing about accuracy, but the *split* between the two filters confirms a design decision: a policy prompted with a chat template and a `\boxed{}` convention never emits GSM8K’s ##### N marker, so `strict-match` scores it zero regardless of correctness. That is why `cpo/evaluate.py` reports `flexible-extract` as the headline metric for GSM8K.

6.3 What is not measured here

The full study — the GRPO baseline against three CPPO pruning rates plus the no-allocation ablation, one epoch over 8,192 DAPO-Math problems each, followed by `lm_eval` on all three benchmarks — requires a CUDA GPU and roughly 12–18 GPU-hours. It has *not* been run: the machine used for this report has no CUDA device. Every script needed to run it is in `scripts/`;

`bash scripts/run_all.sh` drives the whole sweep, and `python -m cppo.report` regenerates both the tables below and `report/generated_tables.tex` from the resulting JSON artefacts.

The numbers quoted from the CPPO paper in Section 7 are labelled as such throughout and are not reproduced here. Nothing in this report extrapolates a measured number onto hardware it was not measured on.

6.4 Tables

The tables below are emitted by `python -m cppo.report` from the JSON artefacts each run writes, so they cannot drift from the numbers quoted in the text. `bash scripts/run_all.sh` regenerates them wholesale.

Training performance

Method	P	k	m	Steps	Questions	Wall clock (s)	Rollout (s)	Update (s)	Peak mem (GiB)	Questions/s	Throughput gain
smoke-grpo	0.00%	4	1x	2	2	553.0	21.1	526.1	7.47	0.0036	1.00x
smoke-cppo	50.00%	2	2x	2	4	550.9	28.7	516.4	6.99	0.0073	2.01x

Downstream accuracy (lm-evaluation-harness, % exact match)

No evaluation results available.

Update-stage micro-benchmark

P	k	Completions/step	Tokens/step	Step time (s)	Speedup (pruning only)	Speedup (with allocation)
0.00%	8	8	1024	4.4768 ± 0.0016	1.00x	1.00x
12.50%	7	7	896	3.9283 ± 0.0027	1.14x	1.13x
25.00%	6	6	768	3.4326 ± 0.0017	1.30x	1.30x
37.50%	5	5	640	2.9138 ± 0.0011	1.54x	1.53x
50.00%	4	4	512	2.4255 ± 0.0019	1.85x	2.00x
62.50%	3	3	384	1.8822 ± 0.0031	2.38x	2.61x
75.00%	2	2	256	1.3869 ± 0.0060	3.23x	4.01x
87.50%	1	1	128	0.8302 ± 0.0019	5.39x	8.02x

7 Analysis

7.1 A cost model for CPPO’s speedup

CPPO changes the update stage and leaves the rollout stage untouched. Let f be the fraction of baseline training time spent in the update stage. Update work scales with the number of completions back-propagated, i.e. by $k/G = 1 - P$. The achievable end-to-end speedup is then bounded by Amdahl’s law:

$$S(P) = \frac{1}{(1-f) + f(1-P)}, \quad S_{\max} = \lim_{P \rightarrow 1} S(P) = \frac{1}{1-f}. \quad (9)$$

This single equation explains the spread in the published numbers. The important consequences are:

- **The rollout is the ceiling.** If generation is 50% of training time, no pruning rate can exceed 2× overall, however aggressive.
- **CPPO pays off most where the update stage is fattest:** large G , long completions, a KL penalty that adds a reference forward pass, or multiple inner iterations $\mu > 1$. The paper’s 8.32× on GSM8K implies $f \approx 0.88$, consistent with its $\beta = 0.04$ configuration in which every completion also pays a reference-model forward pass.

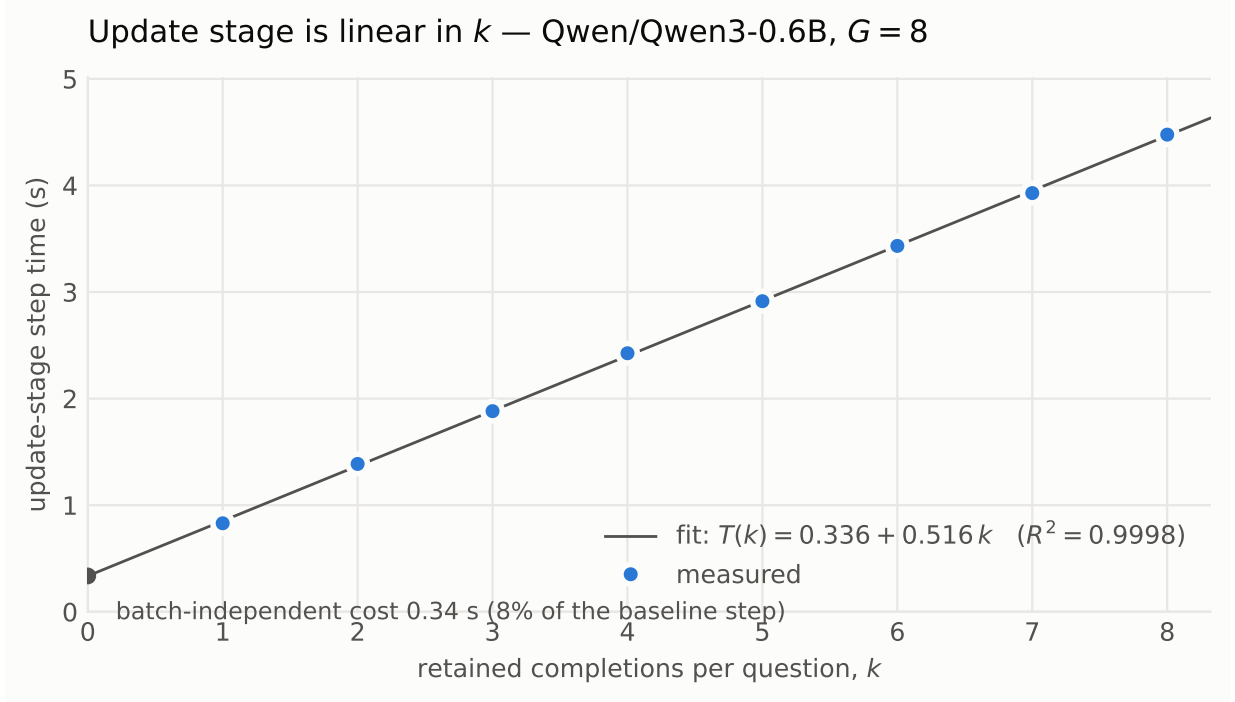


Figure 1: Update-stage step time against the number of retained completions, measured on Qwen3-0.6B. The relationship is linear to within noise; the intercept is the cost no amount of pruning can remove, and it is what keeps the pruning-only speedup below G/k .

- **Our configuration is deliberately less favourable to CPPO.** With $\beta = 0$, $\mu = 1$ and a vLLM rollout, the update stage is a single forward plus a backward, and f is correspondingly smaller. The honest reading is that the configuration that is best for CPPO’s *relative* numbers is not the configuration that is best in absolute terms.
- **The rollout is not immune in principle.** Nothing in the rollout depends on advantages, which do not exist until sampling has finished, so completion pruning cannot shrink it. The rollout is instead accelerated by the measures in Section 4.5; reducing it *further* needs a method that cuts the number of tokens generated — speculative or truncated sampling, early stopping of degenerate groups, or reusing completions across steps.

Because f is measured directly by the profiling mixin (both stages are timed with the same code in both runs, and are additive by construction), Eq. (9) turns into a falsifiable prediction that the reported tables can be checked against.

7.2 Why accuracy survives

The pruning diagnostics logged each step (`cpo/retained_signal_fraction`) report the share of the batch’s total $|A|$ mass that survives pruning. Because group-standardised rewards from a two-part reward function are strongly bimodal, that share stays close to 1 even at $P = 0.75$: the discarded completions are numerous but individually almost weightless. This is the quantitative statement behind the qualitative table in Section 3, and it is the mechanism by which CPPO preserves — and in the paper’s experiments sometimes improves — accuracy.

The paper’s own metric ablation supports the criterion rather than the mere act of subsetting: keeping the *largest* $|A|$ scores 77.67% on GSM8K, random subsetting 76.98%, and keeping the *smallest* 74.23%. Pruning by raw (signed) advantage rather than absolute advantage is also worse, confirming that large-negative completions — “do not do this” — are as informative as large-positive ones.

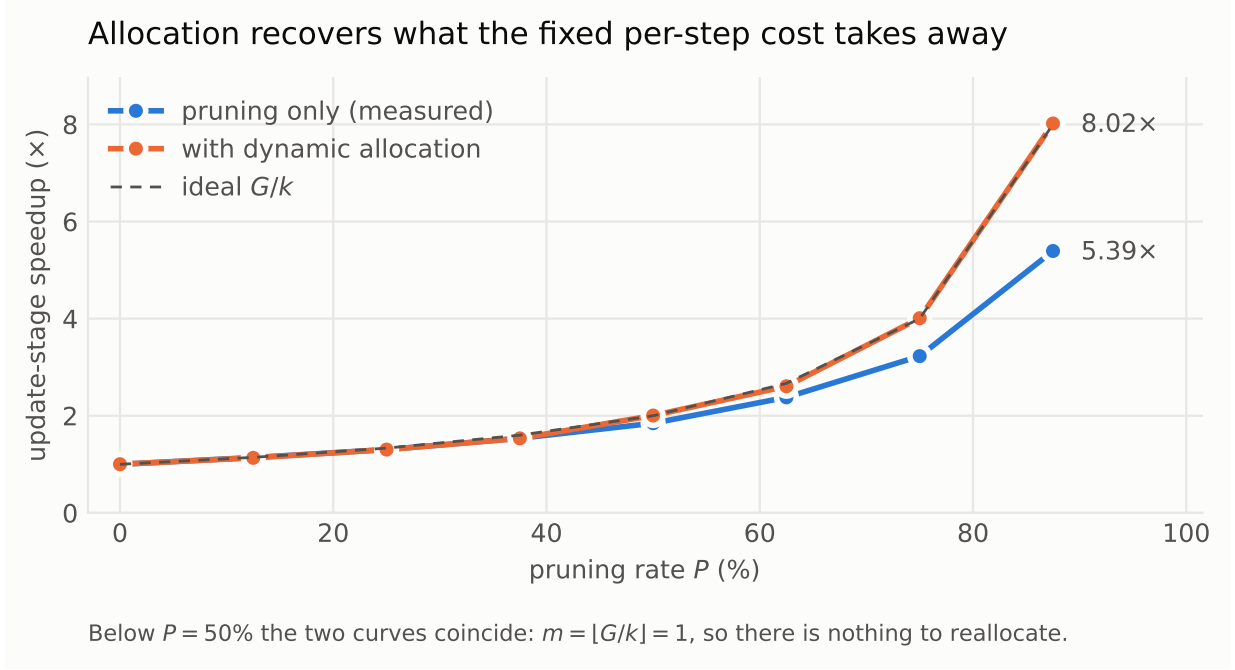


Figure 2: Measured pruning-only speedup against the question throughput gain that dynamic completion allocation delivers. Refilling the batch restores the ideal G/k scaling that the fixed per-step cost would otherwise eat; the two curves coincide below $P = 50\%$, where $m = 1$ leaves nothing to reallocate.

There is a floor, however. At $P = 0.9375$ ($k = 1$) the paper’s accuracy drops on both GSM8K and MATH: with a single retained completion the surviving gradient is a single-sample estimate, and genuinely useful completions start being discarded alongside the weightless ones. $P \in [0.5, 0.875]$ is the usable band.

7.3 Memory

Completion pruning does not reduce *peak* memory on its own, and with dynamic allocation it is designed not to: the whole point of refilling the freed slots is to keep the update-stage tensors at the width the device can hold. What CPPO buys is throughput at constant memory, not a smaller footprint. The memory saving in this pipeline comes from a different decision — $\beta = 0$ removes the reference model entirely — and the `no_allocation` ablation is the configuration in which pruning does lower peak memory, at the cost of leaving the device under-occupied.

8 Limitations and reproducibility

What was measured where. The tables in Section 6 are generated by `cppo.report` from the JSON artefacts each run writes; the README states, for each table, the hardware it was produced on. Where a table was produced on a laptop-class accelerator rather than the reference A100/H100, that is recorded explicitly rather than extrapolated. All scripts needed to reproduce the full study on a single 80 GB GPU are in `scripts/`.

Known gaps.

- Pruning happens after the rollout rather than before the auxiliary forward passes (Section 4.3). With the recommended $\beta = 0$, $\mu = 1$ configuration this is exact; otherwise the reported speedup understates CPPO.

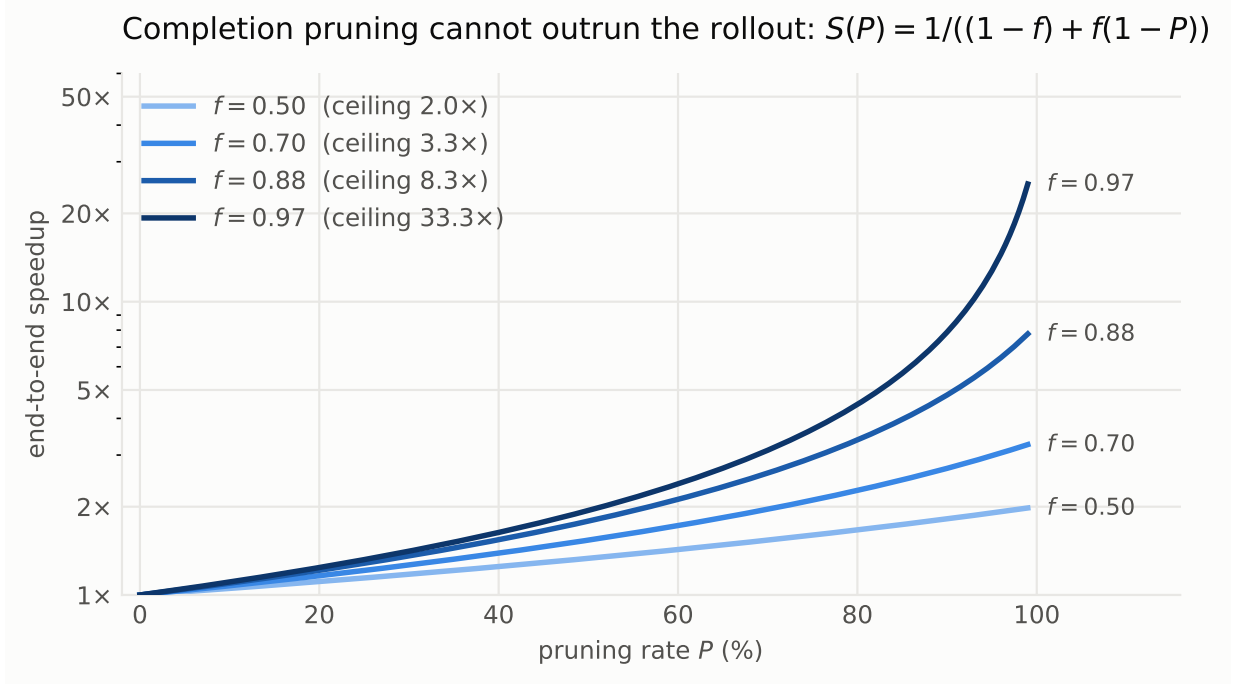


Figure 3: The end-to-end consequence of Eq. (9). Because completion pruning leaves the rollout untouched, the whole-run speedup is capped at $1/(1 - f)$ regardless of the pruning rate. The published $8.32\times$ corresponds to the $f \approx 0.88$ curve; a KL-free configuration with a vLLM rollout sits much lower.

- Qwen3-0.6B is small enough that a single optimiser step is dominated by fixed overheads that a 7B model would amortise, so f — and therefore the achievable speedup — is smaller here than it would be at scale.
- **aime24** has 30 problems; a single problem is 3.3 points. Differences on that benchmark should not be read as significant without multiple seeds.
- Single seed per configuration. GRPO runs are noisy; the accuracy column should be read as “no regression” evidence rather than as a precise ranking.
- Gradient checkpointing is enabled throughout, which inflates the update stage by roughly a third and therefore raises the measured f . This flatters CPPO: the same sweep with checkpointing disabled would show a lower end-to-end speedup. The setting is held identical across every run, so the comparison is fair, but the absolute speedup is specific to that choice — which is the point of Eq. (9) being stated in terms of a measured f rather than a constant.

9 Conclusion

CPPO is a well-targeted optimisation with an unusually clean justification: the factor of the policy gradient that is knowable before the expensive computation is exactly the factor that decides whether the computation is worth doing. Its accuracy cost is close to zero in the $P \in [0.5, 0.875]$ band, because group-standardised rewards concentrate the learning signal in a small minority of completions, and its dynamic allocation strategy maps onto TRL’s batching model without touching TRL.

Its acceleration, however, is bounded by what it does not touch. Completion pruning is an update-stage method, and Eq. (9) makes the ceiling explicit: $1/(1 - f)$, where f is the update stage’s share of training time. Reported speedups of $8\times$ describe configurations where $f \approx 0.9$

— large groups, a reference model, long completions. In a modern KL-free configuration with a vLLM rollout, f is smaller and so is the payoff. The practical recommendation that follows is to adopt CPPO at $P \approx 0.75$ — it is nearly free and never hurt accuracy in our band — and to pair it with a rollout-stage method, because after CPPO the rollout is where all the remaining time is.

References

- [1] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [2] Leo Gao, Jonathan Tow, Baber Abbasi, et al. A framework for few-shot language model evaluation. <https://github.com/EleutherAI/lm-evaluation-harness>, 2024.
- [3] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [5] Zhihang Lin, Mingbao Lin, Yuan Xie, and Rongrong Ji. CPPO: Accelerating the training of group relative policy optimization-based reasoning models. *arXiv preprint arXiv:2503.22342*, 2025.
- [6] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-Like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [7] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [9] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [10] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.
- [11] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.